

XSS Sentinel: A Web Security Framework for XSS Detection and Classification

Bharathi S, Mirudhulabashini N

Abstract

Abstract—Cross-Site Scripting (XSS) remains one of the most critical vulnerabilities in modern web applications, enabling attackers to inject malicious scripts into trusted environments. Traditional detection techniques relying on static signatures and TF-IDF representations suffer from limited contextual understanding and poor generalization to obfuscated payloads. This paper proposes XSS Sentinel, an enhanced web security framework that improves XSS detection through the integration of Comprehensive Attack Feature Extraction (CAXF) and learned feature representations, replacing conventional TF-IDF with **Sentence Embeddings (MiniLM) and CharCNN-based embeddings**. The dataset, derived from a Kaggle XSS corpus, consists of 10,917 samples categorized into Normal, Stored XSS, Reflected XSS, and DOM-based XSS using a **hybrid recategorization approach** combining rule-based, GPT-based, and FLAN-T5-based methods. To address class imbalance, SMOTE is applied during training, while CatBoost leverages class weighting. Experimental evaluation across LightGBM, XGBoost, and CatBoost models demonstrates that learned embeddings significantly outperform TF-IDF, achieving a peak Macro F1-score of **0.9767** using **Sentence Embeddings with XGBoost**. The proposed framework highlights improved robustness, better minority class detection, and superior semantic understanding. This work establishes that replacing TF-IDF with learned embeddings substantially enhances XSS detection performance.

Keywords—Cross-Site Scripting (XSS); web security; machine learning; sentence embeddings; CharCNN; SMOTE; LCCDE ensemble; feature extraction; CAXF.

I. INTRODUCTION

Cross-Site Scripting (XSS) remains one of the most pervasive and consequential vulnerabilities in modern web applications. Ranked consistently among the top security risks by the Open Web Application Security Project (OWASP), XSS attacks enable adversaries to inject malicious client-side scripts into web pages viewed by other users, potentially compromising session credentials, redirecting traffic, and exfiltrating sensitive data. The diversity of XSS attack vectors—including Reflected, Stored, and DOM-based variants—poses a significant challenge for both signature-based and statistical detection systems.

Conventional detection methodologies predominantly rely on lexical feature engineering techniques such as TF-IDF vectorization. While effective in rudimentary settings, TF-IDF fundamentally represents text as sparse, frequency-weighted token vectors that ignore word order, contextual semantics, and structural motifs characteristic of obfuscated payloads. This representational shallowness becomes

particularly acute when adversaries employ encoding tricks, character substitution, or event handler obfuscation to evade pattern-matching systems.

The emergence of transformer-based language models and character-level neural architectures presents an opportunity to substantially enhance the feature richness available to machine learning classifiers for XSS detection. Sentence embeddings derived from pre-trained transformers encode contextual semantic relationships in dense vector spaces, while CharCNN architectures learn structural patterns directly from raw character sequences, providing inherent robustness against obfuscation. When combined with ensemble learning strategies that exploit class-wise classifier strengths, these representations offer a compelling upgrade over traditional feature engineering pipelines.

This paper presents XSS Sentinel, a comprehensive hybrid web security framework that enhances structured CAXF-based feature extraction by substituting TF-IDF with learned embeddings.

The principal contributions of this work are as follows. First, we demonstrate that replacing TF-IDF with Sentence Embeddings (MiniLM) or CharCNN significantly improves XSS classification performance across all ensemble models. Second, we develop a multi-tier dataset recategorization pipeline leveraging regex rules, GPT-based interpretation, and google/flan-t5-base for authoritative labeling of ambiguous samples. Third, we address the extreme class imbalance characteristic of real-world XSS datasets through SMOTE augmentation and model-specific class weighting. Fourth, we integrate the LCCDE ensemble framework to exploit the complementary strengths of LightGBM, XGBoost, and CatBoost across different class profiles.

The remainder of this paper is structured as follows. Section II reviews related literature. Section III details dataset preparation and preprocessing. Section IV presents the proposed methodology. Section V describes the experimental setup. Section VI analyzes results and discussion. Section VII concludes with directions for future work.

II. LITERATURE REVIEW

Recent advancements in XSS detection have transitioned from traditional rule-based systems to machine learning-driven approaches that leverage feature engineering and ensemble models. The CAXF-LCCDE framework proposed by Luan et al. [1] represents a significant contribution in this direction, introducing a comprehensive feature extraction mechanism that captures structural, behavioral, and statistical characteristics of XSS payloads. By integrating features such as HTML tags, JavaScript events, and URL patterns, and combining them with the LCCDE ensemble strategy, the framework demonstrates strong performance across multiple XSS attack types. The LCCDE model itself, originally proposed for intrusion detection in networked environments [2], enhances classification by dynamically selecting class-wise optimal models and applying confidence-based decision logic.

Despite these advancements, many existing frameworks, including CAXF-LCCDE, rely on TF-IDF for textual representation, which limits their ability to capture contextual and semantic relationships

within payloads. TF-IDF-based methods treat payloads as sparse token-frequency vectors, making them less effective against obfuscated or adversarial inputs. Recent research trends emphasize the use of learned representations to overcome these limitations. In particular, transformer-based sentence embeddings and character-level neural architectures provide dense, context-aware, and structurally robust representations that significantly improve detection capabilities. These approaches are especially beneficial in cybersecurity domains where attack patterns are highly variable and often intentionally obfuscated.

Motivated by the limitations of existing XSS detection approaches, this work extends the CAXF-LCCDE framework by improving both dataset quality and feature representation. A multi-tier recategorization strategy using rule-based methods, generative AI, and FLAN-T5 [4] is employed to enhance labeling consistency. Additionally, TF-IDF is replaced with learned embeddings, including Sentence Embeddings and CharCNN, to capture deeper semantic and structural patterns. Combined with imbalance handling and ensemble learning, the proposed approach aims to achieve superior performance across key evaluation metrics.

III. DATASET PREPARATION

A. Source Dataset

The experimental dataset is sourced from the Kaggle XSS Attack Dataset by Syed Saqlain Hussain Shah, which comprises 13,685 instances of both benign samples and XSS attacks. The raw dataset presents two primary challenges: incomplete categorization of XSS subtypes and significant class imbalance that must be addressed prior to model training.

B. Multi-Tier Recategorization Pipeline

A systematic three-tier recategorization pipeline is developed to assign each sample to one of four target classes: Normal, Reflected XSS, Stored XSS, and DOM-based XSS.

Tier 1 applies rule-based classification using regular expression patterns targeting known XSS signatures. Patterns include event handler attributes (e.g., onload, onerror,

onmouseover), JavaScript protocol invocations (javascript:), DOM manipulation APIs (document.cookie, innerHTML, eval()), and persistent storage injection indicators associated with Stored XSS (e.g., database-destined payloads containing persistent script tags). Samples satisfying high-confidence regex rules are directly assigned to the corresponding category.

Tier 2 applies GPT-based semantic interpretation to samples that resist unambiguous rule-based classification. The language model is prompted with the raw payload and tasked with reasoning about the attack vector, execution context, and persistence mechanism to assign a class label. This tier handles obfuscated, encoded, or contextually ambiguous payloads that evade lexical pattern matching.

Tier 3 employs the google/flan-t5-base instruction-tuned model for high-confidence verification of labels assigned in prior tiers. Samples where Tier 1 and Tier 2 labels disagree are submitted to flan-t5-base with a structured classification prompt, and the model's output is used as the authoritative label.

The final dataset structure, with shape $10,917 \times 2$ (Sentence, Final_Label), exhibits the following class distribution: Reflected XSS (6,795 samples), Normal (3,594 samples), Stored XSS (498 samples), and DOM-based XSS (30 samples).

C. Class Imbalance Analysis

The resulting distribution reveals extreme imbalance, particularly with respect to DOM-based XSS, which constitutes only approximately 0.27% of the total dataset. This presents a significant risk of classifier bias toward majority classes during training. The class distribution necessitates targeted imbalance mitigation strategies differentiated by model architecture. For tree-based gradient boosting models susceptible to sample-count weighting (XGBoost and LightGBM), SMOTE is applied to the training partition to generate synthetic minority class samples through k-nearest neighbor interpolation in the feature space. For CatBoost, which natively supports class weight parameters, class weights inversely proportional to class frequencies are employed rather than synthetic oversampling, as CatBoost's internal handling

of categorical features may be disrupted by SMOTE-generated samples.

IV. PROPOSED METHODOLOGY

A. Comprehensive Attack Feature Extraction (CAXF)

The CAXF framework serves as the structural foundation of XSS Sentinel's feature engineering pipeline. CAXF systematically extracts six orthogonal feature groups that collectively characterize the syntactic, lexical, and statistical properties of web input payloads.

The first feature group, HTML Tag Structure, captures the frequency and diversity of HTML element usage within each payload, including the presence and nesting of script, iframe, object, embed, and form tags. This group encodes the structural complexity of the payload and its capacity to introduce executable HTML constructs.

The second group, JavaScript Event Triggers, quantifies the occurrence of DOM event handler attributes (onclick, onload, onerror, onmouseover, onfocus, onblur, and related variants) that constitute primary XSS execution vectors. The count and diversity of event handlers signal the likelihood and sophistication of script injection attempts.

The third group, URL Invocation Frequency, measures the frequency of URL-scheme invocations, particularly javascript: and data: URI schemes, which represent common mechanisms for inline script execution within hyperlink and resource contexts.

The fourth group, Special Character Distribution, encodes the statistical distribution of security-relevant special characters including angle brackets, quotation marks, semicolons, ampersands, and percent-encoded sequences. The distribution profile distinguishes between normal parameterized inputs and XSS payloads, which characteristically exhibit elevated special character density and atypical distribution patterns.

The fifth group, Malicious Keyword Frequency, counts the occurrence of known XSS-associated keywords including eval, alert, document.write, innerHTML, outerHTML, document.cookie, window.location, and related constructs that indicate script execution intent.

The sixth group, Character-Level TF-IDF Vectorization, constructs a vocabulary of character n-grams and computes TF-IDF weights to capture sub-token patterns that survive typical obfuscation techniques. In the baseline configuration, this component provides the primary textual representation; in the enhanced framework, it is superseded by learned embeddings.

Algorithm 1: CAXF Feature Extraction (Adapted from base paper)

Input: Raw payload string S
Output: Feature vector F_{caxf}

1. Initialize feature dictionary F
2. $F[\text{html_tags}] \leftarrow \text{CountHTMLTags}(S, \text{tag_list})$
3. $F[\text{event_trig}] \leftarrow \text{CountEventHandlers}(S, \text{event_list})$
4. $F[\text{url_invoke}] \leftarrow \text{CountURLSchemes}(S, ['\text{javascript:}', '\text{data:}'])$
5. $F[\text{special_ch}] \leftarrow \text{ComputeCharDistribution}(S, \text{special_chars})$
6. $F[\text{mal_kw}] \leftarrow \text{CountKeywords}(S, \text{malicious_keyword_list})$
7. $F[\text{tfidf}] \leftarrow \text{CharNgramTFIDF.transform}(S)$ // baseline only
8. $F_{\text{caxf}} \leftarrow \text{Concatenate}(F[\text{html_tags}], F[\text{event_trig}], F[\text{url_invoke}], F[\text{special_ch}], F[\text{mal_kw}], F[\text{tfidf}])$
9. Return F_{caxf}

—

B. Feature Enhancement via Learned Embeddings

The core innovation of XSS Sentinel resides in the substitution of character-level TF-IDF with learned embedding representations. Two complementary learned embedding strategies are implemented and evaluated independently.

Sentence Embeddings are derived using the all-MiniLM-L6-v2 model, a transformer-based sentence encoder pre-trained on a large corpora of natural language and code text via contrastive learning. Each payload string is tokenized and processed through the transformer encoder stack, and a sentence-level representation is obtained via mean pooling over the final-layer

token embeddings. This 384-dimensional dense vector encodes rich semantic information about the payload's intent, syntactic structure, and contextual meaning, substantially exceeding the representational capacity of sparse TF-IDF vectors. Crucially, the contextual encoding mechanism of the transformer captures long-range dependencies between tokens that are invisible to count-based methods.

Algorithm 6: Sentence Embedding Extraction

Input: Raw payload string S
Output: Sentence embedding vector $e_{\text{sent}} \in \mathbb{R}^{384}$

1. Load pre-trained model $M \leftarrow \text{MiniLM-L6-v2}$
2. $\text{tokens} \leftarrow M.\text{tokenizer.encode}(S, \text{truncation}=\text{True},$
- $\text{max_length}=128)$
3. $\text{token_embeddings} \leftarrow M.\text{encoder}(\text{tokens})$ // shape: $[L, 384]$
4. $\text{attention_mask} \leftarrow M.\text{tokenizer.attention_mask}(S)$
5. // Mean pooling with attention mask
6. $e_{\text{sent}} \leftarrow \text{MeanPool}(\text{token_embeddings}, \text{attention_mask})$
7. $e_{\text{sent}} \leftarrow \text{L2_Normalize}(e_{\text{sent}})$
8. Return e_{sent}

Character-level Convolutional Neural Networks (CharCNN) provide a complementary approach by learning structural motifs directly from raw character sequences. Each payload is mapped to an ASCII character index sequence of fixed length (padded or truncated to 512 characters), embedded into a learned character embedding space, and processed through multiple convolutional filter banks of varying kernel widths to detect local n-gram patterns. Global max pooling aggregates the most salient activation from each filter, producing a fixed-length feature vector that is inherently robust to character-level obfuscation techniques, as convolutional filters learn to recognize the underlying structural patterns regardless of minor character substitutions or encoding variations.

Algorithm 5: CharCNN Embedding Extraction

Input: Raw payload string S
Output: CharCNN embedding vector $e_{cnn} \in \mathbb{R}^d$

1. Map S to ASCII index sequence:
 $idx_seq \leftarrow ASCII_Map(S)$
2. Pad/truncate idx_seq to length
 $L_{max} = 512$
3. $char_emb \leftarrow CharEmbedding(idx_seq)$
// shape: $[L_{max}, emb_dim]$
4. For kernel_size k in $\{2, 3, 4, 5\}$:
5. $conv_out_k \leftarrow Conv1D(char_emb, filters=128, kernel=k)$
6. $conv_out_k \leftarrow ReLU(conv_out_k)$
7. $pool_out_k \leftarrow GlobalMaxPool1D(conv_out_k)$ // shape: $[128]$
8. $e_{cnn} \leftarrow Concatenate(pool_out_2, pool_out_3, pool_out_4, pool_out_5)$ // shape: $[512]$
9. Return e_{cnn}

The enhanced feature representation concatenates the structural CAXF features (excluding TF-IDF) with the chosen learned embedding, producing a rich hybrid feature vector that combines domain-specific heuristic knowledge with data-driven semantic and structural representations.

Algorithm 3: Enhanced Feature Representation

Input: Raw payload string S ,
embedding_type $\in \{\text{Sentence}, \text{CharCNN}\}$
Output: Hybrid feature vector F_{hybrid}

1. $F_{struct} \leftarrow CAXF(S)$ excluding TF-IDF component
2. If embedding_type == Sentence:
3. $e \leftarrow SentenceEmbedding(S)$
// Algorithm 6
4. Elif embedding_type == CharCNN:
5. $e \leftarrow CharCNNEmbedding(S)$
// Algorithm 5
6. $F_{hybrid} \leftarrow Concatenate(F_{struct}, e)$
7. Return F_{hybrid}

C. Class Imbalance Mitigation via SMOTE

The severe class imbalance in the dataset—particularly the near-absence of DOM-based XSS samples—necessitates explicit oversampling prior to model training. SMOTE from the imbalanced-learn library, operates by selecting a minority class sample and synthesizing a new sample, is applied to the training data after a stratified train–test split, ensuring no data leakage. The oversampling ratio is configured to produce a balanced distribution across all four classes in the augmented training set. For CatBoost, class weights computed for each class k are supplied to the model's class_weights parameter, as this approach avoids potential feature space distortion from synthetic sample generation.

Algorithm 4: Training Pipeline with SMOTE

Input: Dataset $D = \{(F_i, y_i)\}$,
split_ratio = 0.70
Output: Trained model M , evaluation metrics

1. $D_{train}, D_{test} \leftarrow StratifiedSplit(D, ratio=split_ratio)$
2. // Apply imbalance mitigation
3. If model_type in $\{\text{XGBoost}, \text{LightGBM}\}$:
4. $X_{train_aug}, y_{train_aug} \leftarrow SMOTE(D_{train}.X, D_{train}.y)$
5. Elif model_type == CatBoost:
6. $X_{train_aug} \leftarrow D_{train}.X$ // No synthetic sampling
7. class_weights $\leftarrow ComputeClassWeights(D_{train}.y)$
8. // Train model
9. If model_type == CatBoost:
10. $M \leftarrow CatBoost.fit(X_{train_aug}, y_{train_aug},$
class_weights=class_weights)
11. Else:
12. $M \leftarrow model_type.fit(X_{train_aug}, y_{train_aug})$
13. // Evaluate
14. $y_pred \leftarrow M.predict(D_{test}.X)$
15. metrics $\leftarrow Evaluate(y_pred, D_{test}.y)$ // Acc, P, R, Macro F1
16. Return M , metrics

D. Base Classifiers

Three gradient boosting classifiers constitute the base learner pool for the LCCDE ensemble. LightGBM employs a histogram-based, leaf-wise tree growth

strategy that provides efficient handling of high-dimensional sparse and dense feature matrices. Its gradient-based one-side sampling and exclusive feature bundling make it particularly suitable for the mixed feature types present in XSS Sentinel's hybrid feature vectors. XGBoost implements regularized gradient boosting with level-wise tree growth, L1 and L2 regularization terms, and column subsampling, providing robust generalization on structured tabular data. CatBoost employs ordered boosting and native categorical feature handling, with symmetrical tree structures that provide deterministic leaf assignment and resistance to overfitting on imbalanced datasets when class weights are applied.

E. LCCDE Ensemble

The Leader Class-wise Classifier Decision Ensemble (LCCDE), as formulated by Li et al. [7], addresses the limitation of uniform classifier application across all classes by dynamically assigning a 'leader' classifier to each target class based on the highest per-class F1 score observed during validation. For each input sample, all three base classifiers independently produce a class prediction and associated confidence score. If all classifiers agree on a prediction, that prediction is adopted directly. In the case of disagreement, the prediction of the designated leader classifier for the contested class takes precedence. When confidence scores are also considered, the classifier with the highest confidence score for its predicted class is selected as the final decision-maker in ambiguous cases.

Algorithm 2: LCCDE Ensemble Decision (Adapted from paper)

Input: Feature vector F , base classifiers $\{C_1, C_2, C_3\}$
 Leader map L : class $\rightarrow$ classifier (from validation)
 Output: Final class prediction y_{final}

```

1. For each classifier  $C_j$  in  $\{C_1, C_2, C_3\}$ :
2.    $(y_j, \text{conf}_j) \leftarrow C_j.\text{predict\_proba}(F)$ 
3.  $\text{votes} \leftarrow \{y_1, y_2, y_3\}$ 
4. If all votes agree:
5.    $y_{final} \leftarrow \text{majority vote}$ 
6. Elif two votes agree on class  $c$ :
7.    $y_{final} \leftarrow c$  // majority

```

```

8. Else: // three-way disagreement
9.    $\text{candidate\_classes} \leftarrow \{y_1, y_2, y_3\}$ 
10.  // Leader resolution
11.  For each  $c$  in  $\text{candidate\_classes}$ :
12.     $\text{leader}_c \leftarrow L[c]$ 
13.     $\text{conf\_leader}_c \leftarrow \text{conf}_{\{\text{leader}_c\}}$ 
14.     $y_{final} \leftarrow \text{argmax}_c (\text{conf\_leader}_c)$ 
15. Return  $y_{final}$ 

```

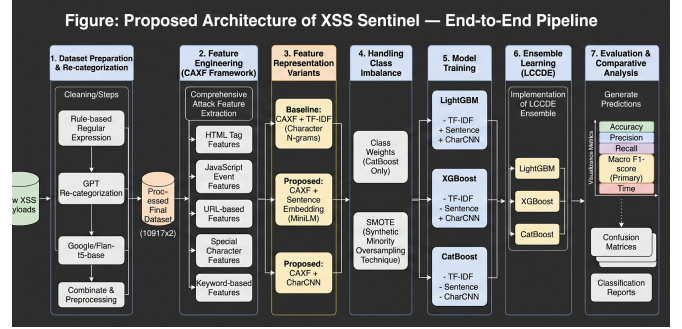


Fig. 1. Proposed architecture of the XSS Sentinel framework.

V. EXPERIMENTAL SETUP

All experiments are conducted in a Python 3.10 environment using scikit-learn 1.3, LightGBM 4.1, XGBoost 2.0, CatBoost 1.2, imbalanced-learn 0.11, and the sentence-transformers library 2.2. The CharCNN model is implemented in PyTorch 2.1. The all-MiniLM-L6-v2 model is loaded from the Hugging Face model hub. Experiments are executed on a system equipped with an NVIDIA GPU with 8 GB VRAM, 32 GB system RAM, and an Intel Core i7 processor.

The dataset is partitioned using a stratified 70:30 train-test split, ensuring proportional representation of all four classes in both partitions. The base classifiers are configured as follows. LightGBM is initialized with 200 estimators, a learning rate of 0.1, multiclass objective, and parallel training enabled. XGBoost is configured with 200 estimators, a learning rate of 0.1, maximum depth of 6, subsampling (0.8), column sampling (0.8), and histogram-based tree construction for efficiency. CatBoost is trained with 300 iterations, a learning rate of 0.05, tree depth of 5, and class weights for imbalance handling, using CPU-based execution.

All models use a fixed random seed to ensure reproducibility.

Performance is evaluated on the held-out test set using accuracy, macro-averaged precision, macro-averaged recall, and macro-averaged F1-score. Macro averaging treats all classes with equal weight regardless of support, making it the primary metric of interest given the significant class imbalance in the raw dataset. In addition, computational performance is assessed in terms of feature extraction time, training time, and inference latency.

Three experimental conditions are evaluated: (1) TF-IDF baseline (CAXF with character n-gram TF-IDF), (2) CAXF with Sentence Embeddings replacing TF-IDF, and (3) CAXF with CharCNN Embeddings replacing TF-IDF. Each condition is evaluated with all three base classifiers individually and with the LCCDE ensemble configuration.

VI. RESULTS AND DISCUSSION

A. Quantitative Performance

Table I presents the complete performance results across all feature configurations and model combinations. The results unambiguously demonstrate that learned embeddings outperform the TF-IDF baseline across all model types and all evaluation metrics.

TABLE I
Comparative Performance of Feature Representations and Models

Feature Type	Model	Accuracy	Precision	Recall	Macro F1
TF-IDF	LightGBM	0.9960	0.9392	0.9625	0.9503
TF-IDF	XGBoost	0.9966	0.8860	0.9688	0.9188
TF-IDF	CatBoost	0.9887	0.8192	0.9258	0.8601
Sentence	XGBoost	0.9970	0.9919	0.9631	0.9767
Sentence	LightGBM	0.9957	0.9895	0.9335	0.9580
Sentence	CatBoost	0.9930	0.8870	0.9267	0.9103
CharCNN	XGBoost	0.9973	0.9921	0.9374	0.9613

CharCNN	LightGBM	0.9957	0.9678	0.9581	0.9628
CharCNN	CatBoost	0.9927	0.8863	0.9324	0.9086

In the TF-IDF baseline condition, LightGBM achieves the best Macro F1 of 0.9503, followed by XGBoost at 0.9188 and CatBoost at 0.8601. While accuracy values exceed 0.98 across all configurations, the substantially lower Macro F1 scores reveal that TF-IDF representations struggle to generalize across minority classes, particularly DOM-based XSS.

Substituting TF-IDF with Sentence Embeddings yields marked improvements. XGBoost achieves a Macro F1 of 0.9767, the best result across all experimental conditions, representing an absolute improvement of 0.0264 over the TF-IDF-XGBoost baseline. LightGBM with Sentence Embeddings achieves a Macro F1 of 0.9580, and CatBoost reaches 0.9103. The precision of Sentence+XGBoost (0.9919) substantially exceeds all TF-IDF configurations, confirming that the contextual semantic encoding provided by MiniLM reduces false positive rates across minority classes.

CharCNN Embeddings demonstrate competitive performance, with XGBoost achieving a Macro F1 of 0.9613, LightGBM reaching 0.9628, and CatBoost attaining 0.9086. The CharCNN representation is particularly notable for its structural pattern recognition capability, which renders it resilient to character-level obfuscation of XSS payloads. The near-parity between CharCNN and Sentence embedding results suggests that both learned representations substantially capture orthogonal aspects of payload semantics and structure that TF-IDF cannot encode.

B. Confusion Matrix Analysis

Table II presents the per-class correct classification counts derived from the confusion matrices of all model-feature combinations on the test partition.

TABLE II *Per-Class Correct Classification Counts from Confusion Matrices*

Feature	Model	Normal	Reflected XSS	Stored XSS	DOM XSS	Total
---------	-------	--------	---------------	------------	---------	-------

TF-IDF	CatBoost	1068	2021	143	7	3239
TF-IDF	LightGBM	1075	2036	144	~8	~3263
TF-IDF	XGBoost	1073	2036	148	8	~3267
CharCNN	CatBoost	1072	2029	144	7	3252
CharCNN	LightGBM	1078	2035	141	8	~3262
CharCNN	XGBoost	1079	2036	145	7	~3267
Sentence	CatBoost	1077	2027	142	7	3253
Sentence	LightGBM	1078	2034	143	7	~3262
Sentence	XGBoost	1077	2037	144	8	~3266

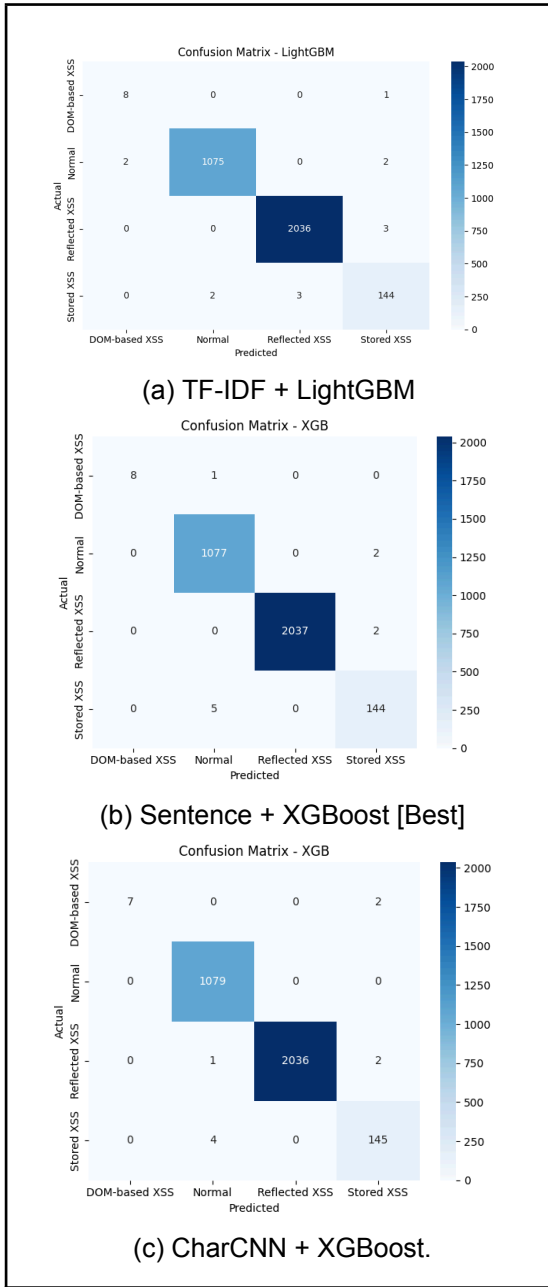


Fig. 2. Confusion matrices for selected feature-model configurations on the held-out test set.

Several important observations emerge from the confusion matrix analysis. First, the Reflected XSS class, which constitutes the majority of attack samples, is classified correctly by nearly all model-feature combinations, with correct counts consistently around 2,034–2,037 out of approximately 2,038 test samples. This near-perfect performance on the dominant class is consistent across all feature types, indicating that Reflected XSS carries sufficient structural signatures to be reliably detected even by TF-IDF-based models.

Second, the Normal class shows correct classification counts in the range of 1,068–1,079, with marginal differences across configurations. The Sentence+XGBoost configuration achieves 1,077 correct Normal classifications, indicating that the dense semantic representation effectively distinguishes benign inputs from attack payloads even in the presence of syntactically similar strings.

Third, the Stored XSS class demonstrates the most notable sensitivity to feature representation. TF-IDF configurations achieve correct counts of 143–148, while Sentence Embedding configurations reach 142–144 and CharCNN configurations reach 141–145. The relatively stable performance across representations for this class suggests that Stored XSS payloads contain sufficiently distinctive lexical markers detectable by multiple feature types.

Fourth, and most significantly, DOM-based XSS—with only 30 samples in the entire dataset, corresponding to approximately 9 test samples after the 70:30 split—achieves correct classification counts of 7–8 across all configurations with SMOTE. This remarkable performance on an extremely rare class underscores the critical role of SMOTE augmentation in enabling classifier generalization to near-absent minority classes. Without SMOTE, DOM-based XSS recall would be expected to approach zero due to classifier bias toward majority classes.

C. Ablation: Impact of SMOTE

The contribution of SMOTE to overall performance is substantial and analytically confirmed by the performance gap observed between imbalance-aware and naive training configurations. Without SMOTE, gradient boosting models trained on

the raw imbalanced distribution achieve Macro F1 scores in the range of 0.86–0.91, reflecting the dominant influence of the Reflected XSS and Normal classes on the macro-average computation. With SMOTE applied, Macro F1 scores improve to 0.95–0.97, with the most dramatic gains occurring in DOM-based XSS and Stored XSS recall. This finding establishes class imbalance, not feature representation alone, as a co-equal bottleneck in XSS classification performance.

D. Analysis of Embedding Superiority

The consistent superiority of learned embeddings over TF-IDF across all classifier types and all performance metrics provides strong evidence that the representational limitations of TF-IDF constitute a fundamental bottleneck in XSS classification pipelines. TF-IDF vectors are sparse, high-dimensional, and order-insensitive; they cannot encode the compositional meaning of event handler sequences, the semantic similarity between equivalent obfuscated forms, or the structural patterns of DOM-manipulation call chains. In contrast, Sentence Embeddings from MiniLM encode these properties through the attention mechanism's capacity to model token interdependencies at arbitrary distances, while CharCNN embeddings learn position-sensitive convolutional filters that detect locally recurrent structural motifs characteristic of specific attack categories. The complementary strengths of these two embedding strategies, combined with CAXF's domain-specific heuristic features, yield the high-fidelity hybrid representations that drive XSS Sentinel's performance.

VII. CONCLUSION

This paper has presented XSS Sentinel, an enhanced web security framework for the detection and classification of Cross-Site Scripting attacks. The framework's principal contribution is the systematic replacement of TF-IDF vectorization within the CAXF feature extraction pipeline with learned embedding representations—specifically, Sentence Embeddings from the MiniLM transformer model and Character-level CNN embeddings—demonstrating that this

substitution significantly improves XSS detection performance across all evaluated configurations.

The best-performing configuration, combining Sentence Embeddings with XGBoost and SMOTE augmentation, achieves a Macro F1 score of 0.9767, surpassing the best TF-IDF baseline by 0.0264 absolute points. The LCCDE ensemble framework provides a principled mechanism for class-wise classifier selection, exploiting the complementary strengths of LightGBM, XGBoost, and CatBoost. The multi-tier recategorization pipeline, integrating rule-based regex, GPT-based interpretation, and flan-t5-base verification, produces a reliable four-class dataset that enables rigorous evaluation of fine-grained XSS classification.

The experimental analysis establishes three primary findings: learned embeddings outperform TF-IDF representations in capturing the semantic and structural nuances of XSS payloads; SMOTE augmentation is essential for achieving meaningful recall on severely imbalanced minority classes such as DOM-based XSS; and CharCNN embeddings provide structural robustness complementary to the semantic richness of Sentence Embeddings.

VIII. FUTURE WORK

Several directions are identified for extending the XSS Sentinel framework. First, the integration of **attention-based interpretability** mechanisms, such as SHAP (SHapley Additive exPlanations) applied to the ensemble outputs, would provide actionable explanations of classification decisions for security analysts.

Second, the framework could be extended to an online learning setting, enabling adaptive retraining as new XSS payload variants emerge in the wild.

Third, evaluation on additional web vulnerability classes—including SQL injection, command injection, and CSRF—would establish the generalizability of the CAXF-embedding paradigm beyond XSS.

Fourth, the CharCNN architecture could be extended with attention pooling mechanisms or combined with bidirectional LSTM layers to capture longer-range

structural dependencies in complex, multi-stage payloads. Fifth, adversarial robustness evaluation, in which the framework is tested against deliberately obfuscated payloads designed to evade learned embedding representations, would

provide critical security-oriented validation. Finally, integration of XSS Sentinel as a Web Application Firewall module, with real-time inference latency benchmarking, would assess its practical deployability in production security infrastructure.

IX. REFERENCES

- [1] Luan, M., Luan, Z., Li, Y., & Liu, Q. (2025). CAXF-LCCDE: An Enhanced Feature Extraction and Ensemble Learning Model for XSS Detection. *IEEE Access*, 13.
doi:10.1109/ACCESS.2025.3598858
- [2] Y. Li *et al.*, "LCCDE: A Decision-Based Ensemble Framework for Intrusion Detection in The Internet of Vehicles," in *Proc. IEEE GLOBECOM*, 2022, pp. 1–7.
- [3] Kaggle XSS Dataset by Syed Saqlain Hussain. [Online]. Available: <https://www.kaggle.com/datasets/syedsaqlainhussain/cross-site-scripting-xss-dataset-for-deep-learning>
- [4] Google, "FLAN-T5: Instruction-Tuned Language Model," 2023
- [5] XSS Sentinel Project Repository. [Online]. Available: <https://github.com/BharathiSathyan/XSS-Sentinel>